wellcome centre integrative neuroimaging

# INTRODUCTION TO GIT

**Juju** Fars
(They/Them/Theirs)
Postdoctoral Research Fellow – Vision group
Open Science Ambassador

juju.fars@ndcn.ox.ac.uk

GitLost?

GitFound!

Gitgood!

# Git/Github/Gitlab

- Who is using Git?

- Who knows the difference between Git, Github and Gitlab?

# Git/Github/Gitlab

What is Git?

Git is a popular version control system.

It is used for:

- Tracking code changes
- Tracking who made changes
- Coding collaboration


"Don't worry, no one really knows how to use git."
*Cassandra Gould van Praag*

# Git/Github/Gitlab


What Git can do:

Manage projects using Repositories

Clone an online repository to work on a local copy

Control and track changes with Staging and Committing

Branch and Merge to allow for work on different parts and versions of a project

Pull the latest version of the project to a local copy

Push local updates to the main project

https://www.w3schools.com/git/git_intro.asp?remote=github

# Git/Github/Gitlab

## Why Git?

- Over 70% of developers use Git!
- Developers can work together from anywhere in the world.
- Developers can see the full history of the project.
- Developers can revert to earlier versions of a project.

## What is GitHub?

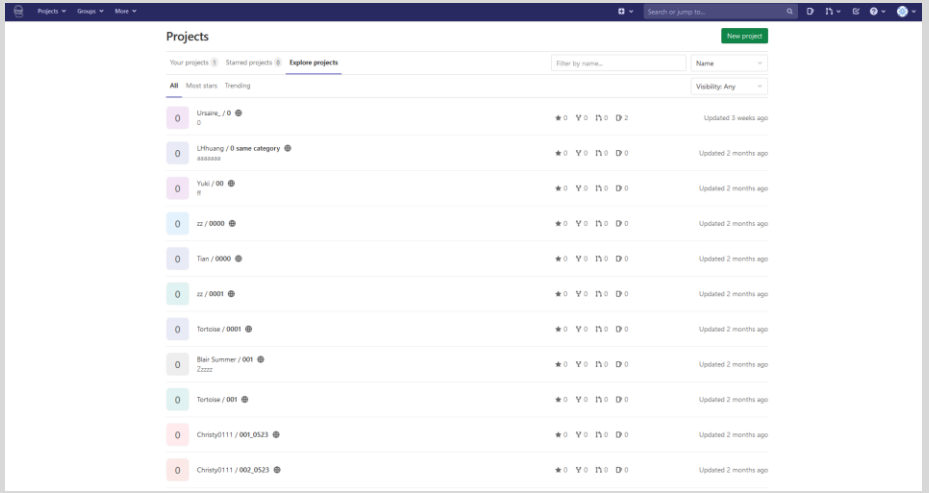
- Git is not the same as GitHub.
- GitHub makes tools that use Git.
- GitHub is the largest host of source code in the world.
- In this tutorial, we will focus on using Git with GitHub.

https://www.w3schools.com/git/git_intro.asp?remote=github

# Git/Github/Gitlab

What is GitLab?

- GitLab makes tools that use Git.
- Gitlab is largely used by our department
- Gitlab can also be used to generate and host webpages (WIN community pages)
- Gitlab also contains all the useful tools found in Github but is dedicated to a specific task/domain/entity (e.g. WIN)




For example: Pavlovia is using Gitlab

https://www.w3schools.com/git/git_intro.asp?remote=github

# Reminder

When using Github or Gitlab, you are using **Git**. **Git** is a tool and Github and Gitlab are interfaces.

## Git (tool)

- Takes care of version control
- One instead of multiple file versions
- You can go back to previous versions

### Bitbucket

Like Github but with a commercial approach

### GitHub (interface)

- Microsoft

### GitLab (interface)

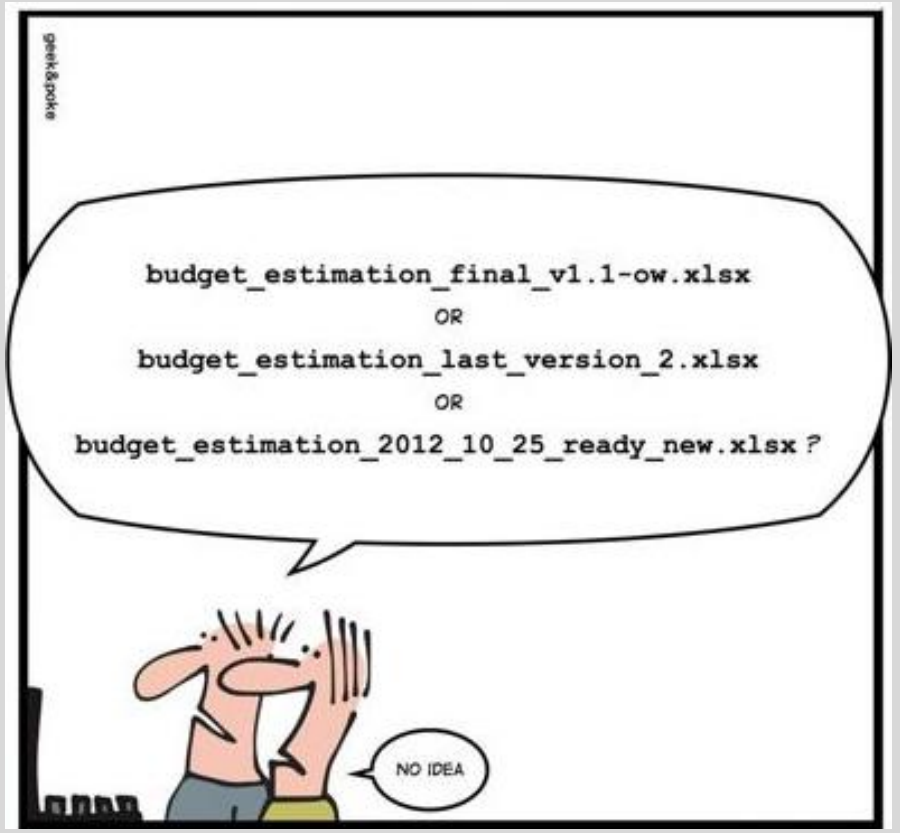
- Hosted by an entity (e.g. WIN)

# Git/Github/Gitlab

## ↳ What is version control?

It records changes to a file or set of files over time so that you can recall specific versions later.

It allows you to:
- revert selected files OR the entire project back to a previous state
- compare changes over time
- see who last modified something that might be causing a problem (who introduced an issue and when).




Version control can be local and online

https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control

# Git/Github/Gitlab

Working with Git

Initialize Git on a folder, making it a Repository

Git now creates a hidden folder (.git) to keep track of changes in that folder

When a file is changed, added or deleted, it is considered modified

You select the modified files you want to Stage

The Staged files are Committed, which prompts Git to store a permanent snapshot of the files

Git allows you to see the full history of every commit.

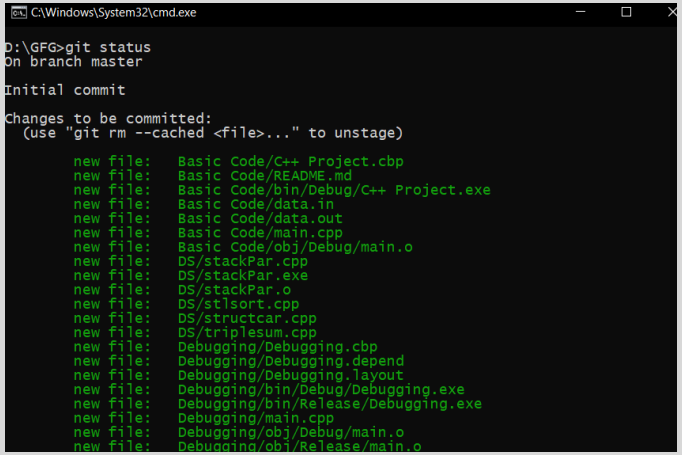
You can revert back to any previous commit.

Git does not store a separate copy of every file in every commit but keeps track of changes made in each commit! (version control)

https://www.w3schools.com/git/git_intro.asp?remote=github

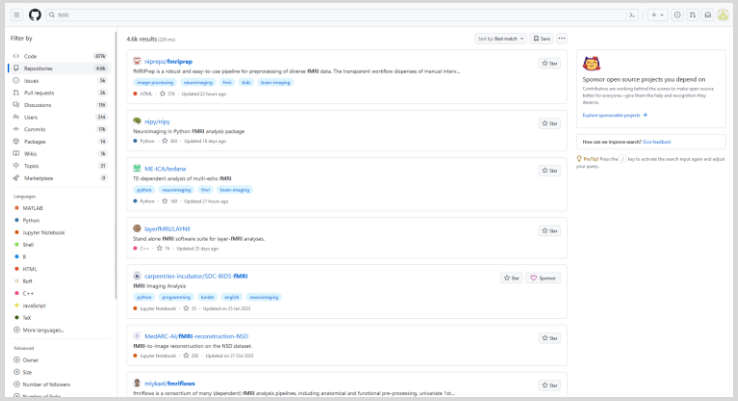
# How to use Git/Github/Gitlab?

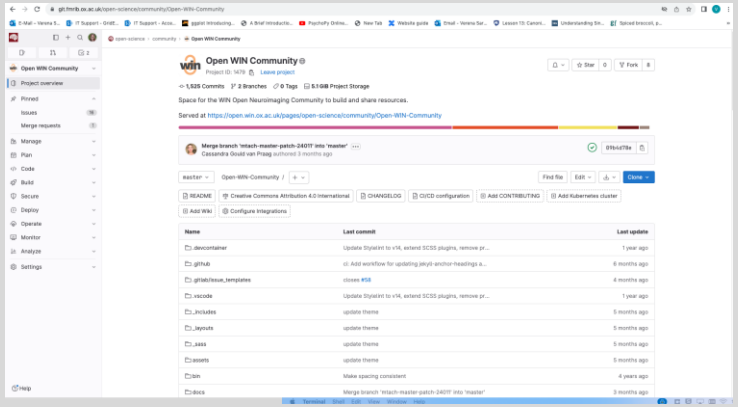
- Terminal (Git)



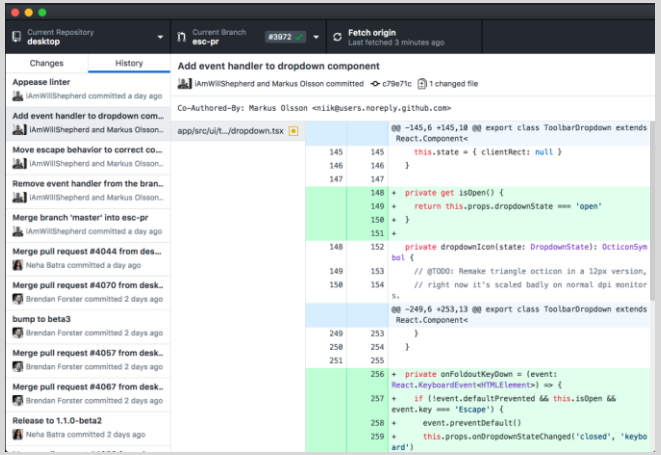

(On windows you can use Gitbash, a git-dedicated terminal)

- Website (Github and Gitlab)



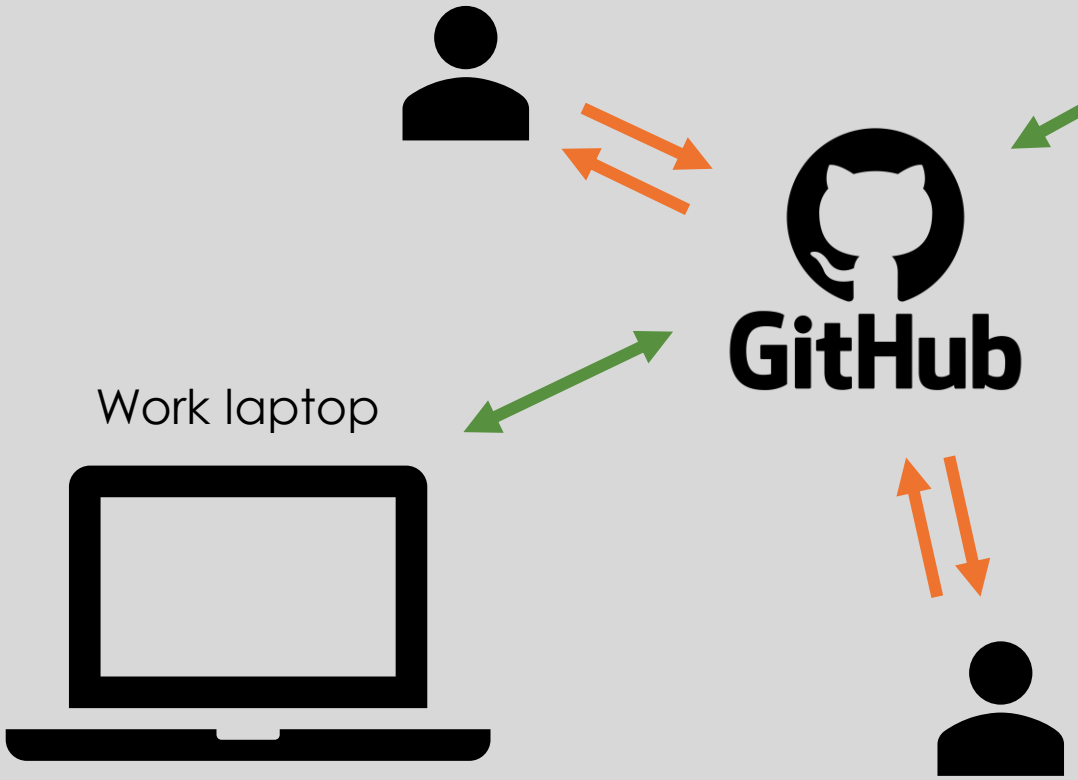



- Apps




Dozens of apps can be used with Git! (e.g. Github Desktop)

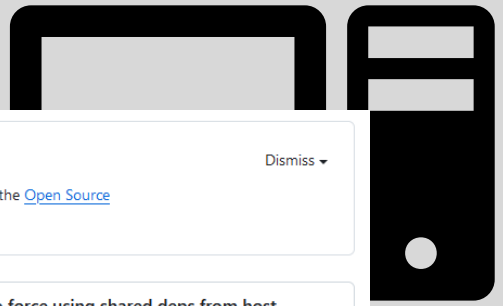
# Practical example



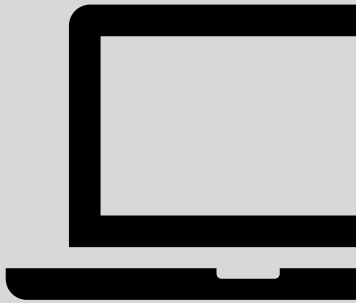

- Synchronisation between remote and online repository
- You can give access to collaborators (internal and external)
- Collaborators...
  - can access the online repository and download a remote version
  - can view and edit the files
  - have access to the latest version (version control)
  - can track and resolve "issues" (tasks that need to be completed)
- Contributions are transparent
- Repositories can be private or public
- Also: useful way of backing up your data

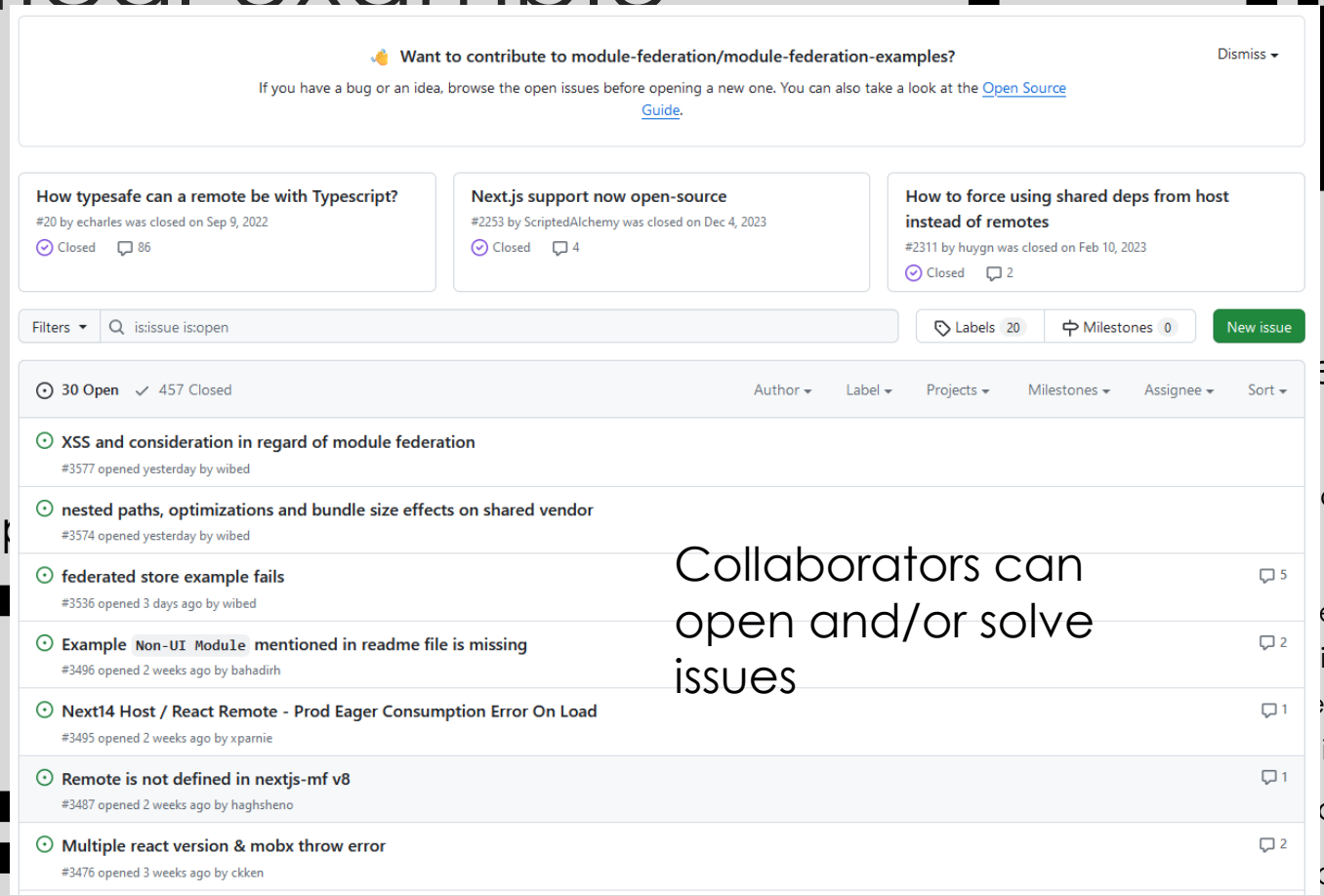
# Practical example

Experimental setup




Work laptop






Collaborators can open and/or solve issues

etween remote and online

collaborators (internal and external)

epository and download a remote version
iles
st version (version control)
issues" (tasks that need to be completed)
arent
ate or public

- Also: useful way of backing up your data

# Git – Lingua Franca

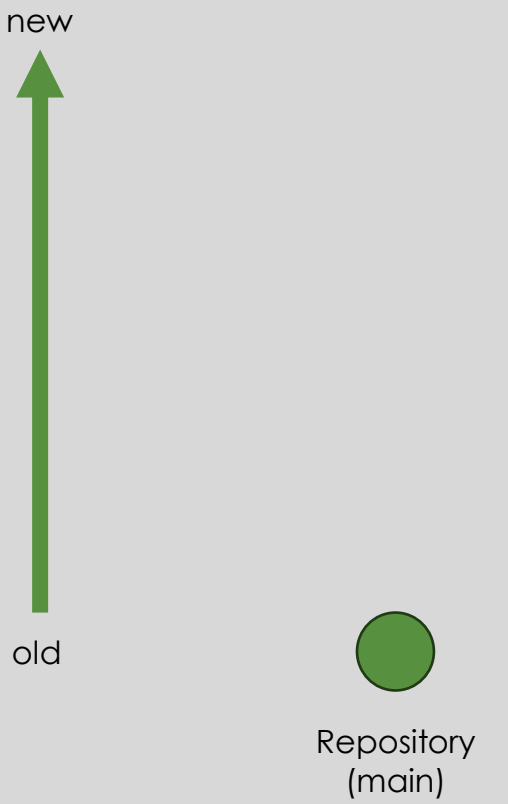
Commands:
$ mkdir myproject
$ cd myproject
$ git init

- **Repository**

If you have a project directory that is currently not under version control and you want to start controlling it with Git, you first need to go to that project's directory. (command cd)

Then use the command "git init"

This creates a new subdirectory named .git that contains all of your necessary repository files — a Git repository skeleton.



# Git – Lingua Franca

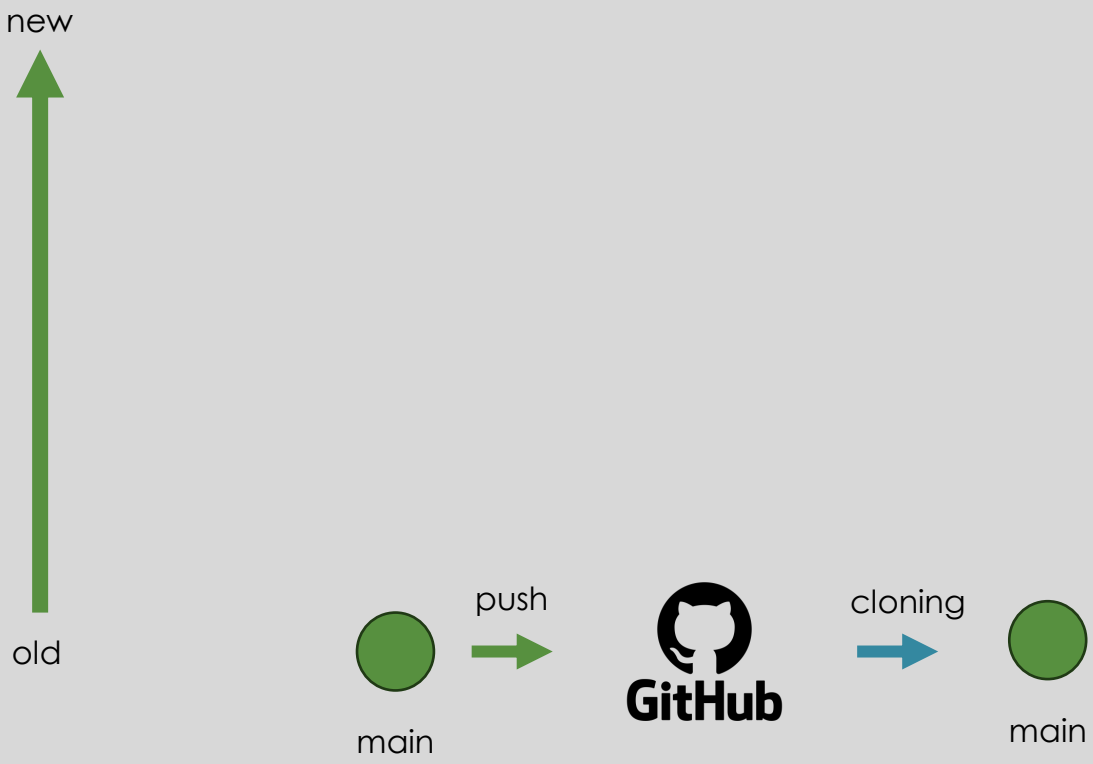
Commands:
$ git push origin
$ git fetch origin
$ git merge origin/master

- **Push** and **Clone**

  If Git is paired with your Github account, you can **Push** your local repository to an online one

  If you want to get a copy of an existing Git repository — for example, a project you'd like to contribute to — or copy your online repository to a local one, the command you need is "git clone".

  For example, you clone an existing repository from Github to a computer



# Git – Lingua Franca

Commands:
$ git commit -m "First commit"

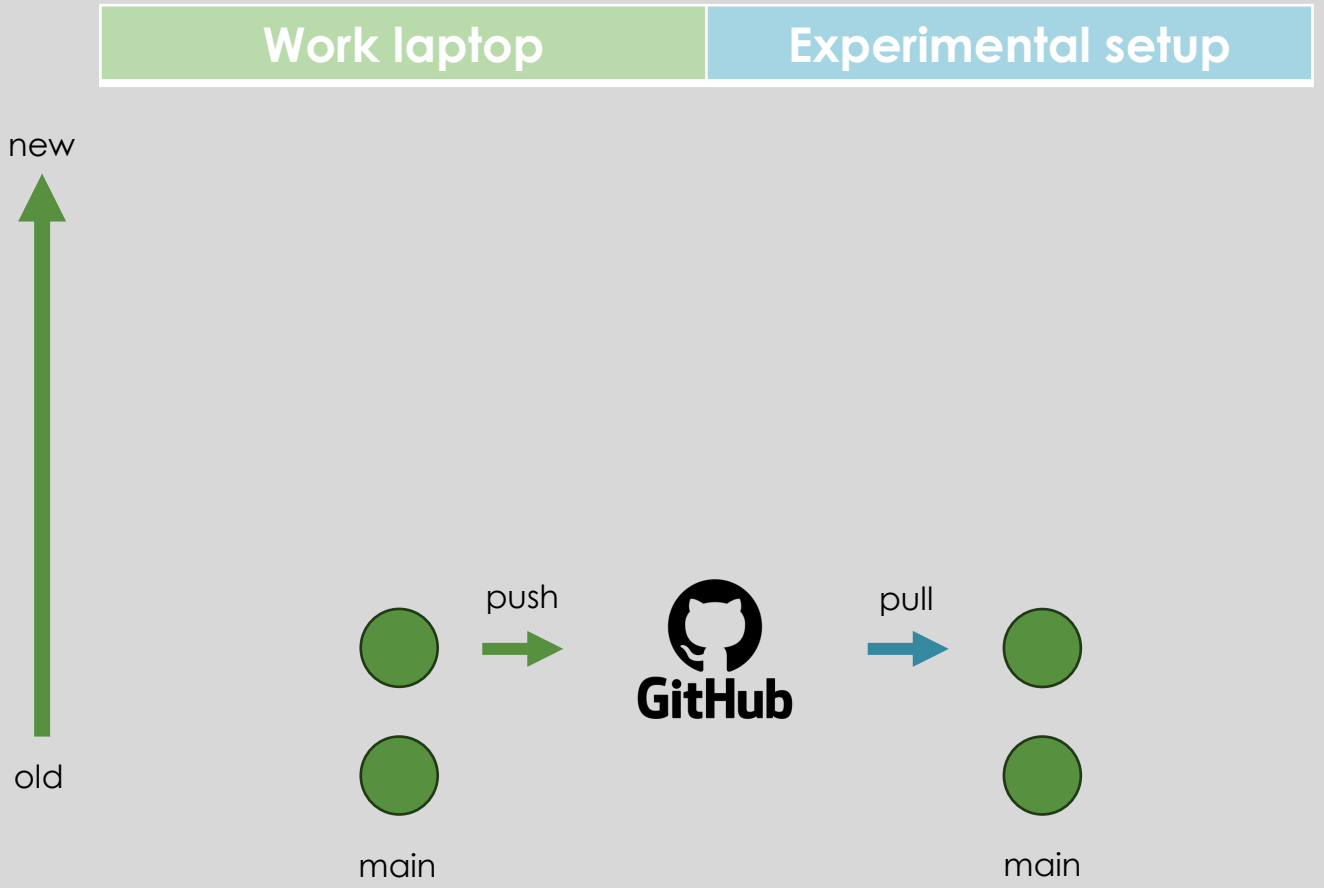
- **Push** and **Pull** and **Commit**

  When you make changes to a folder (you **Commit**), the changes are **tracked**.
  You create a new version of the "main" repository. (you can still go back to the former version)

  You can **Push** the changes on the online repository.

  Then we **Pull** the changes from the online repository to the local experimental setup.

  Pull is different from Clone, because the repository already exists on the local machine!



# Git – Lingua Franca

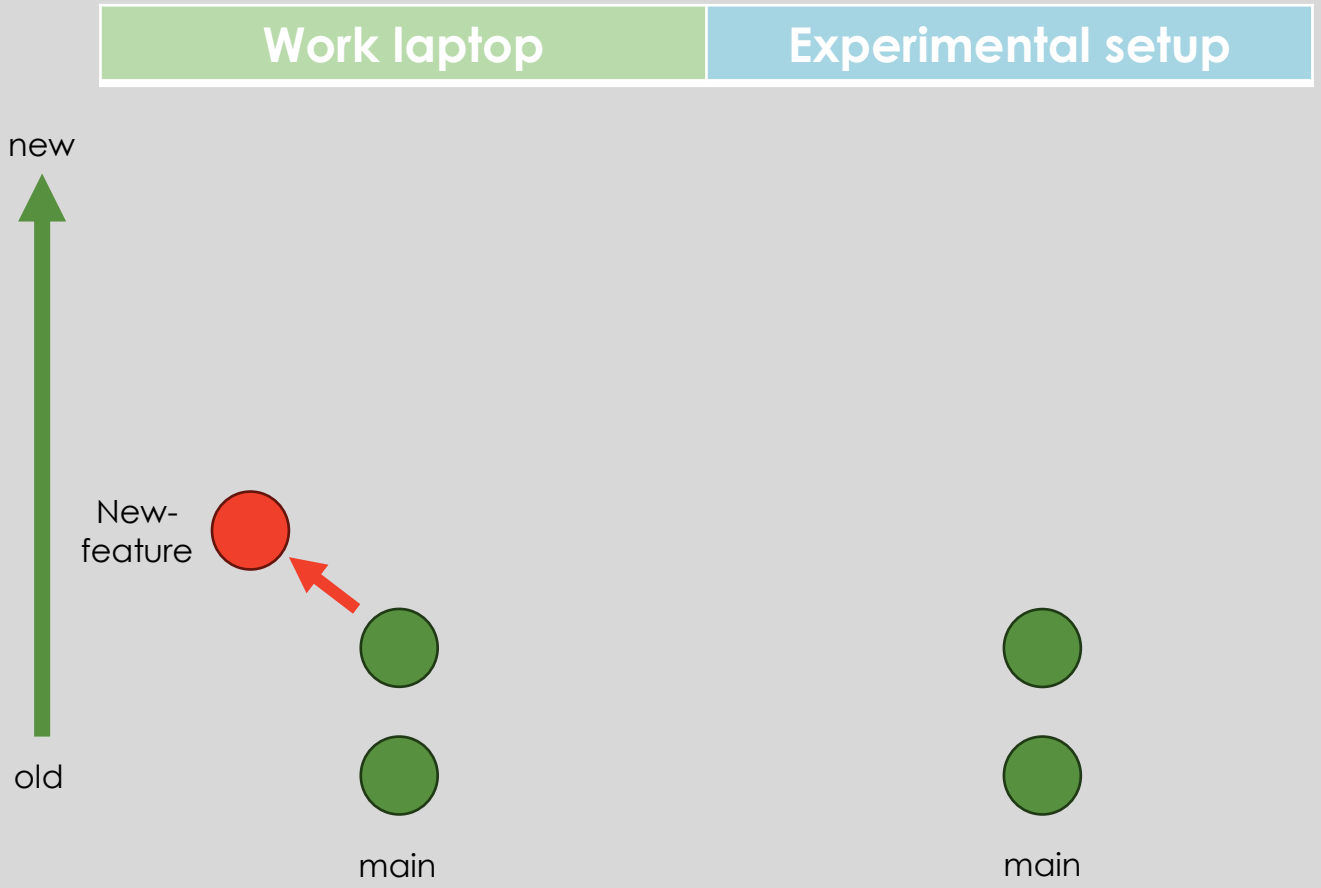
Commands:
$ git branch New-feature
$ git branch
$ git checkout

- **Branch** and **Switch**

If you want to add changes to your folder but keep the main folder intact, you can **Branch**

Both lines of development (main and "new feature") are kept and accessible!
You can **Switch** from one to another at any time (git checkout).

You can also check how many branches you have in your current repository (git branch)



# Git – Lingua Franca

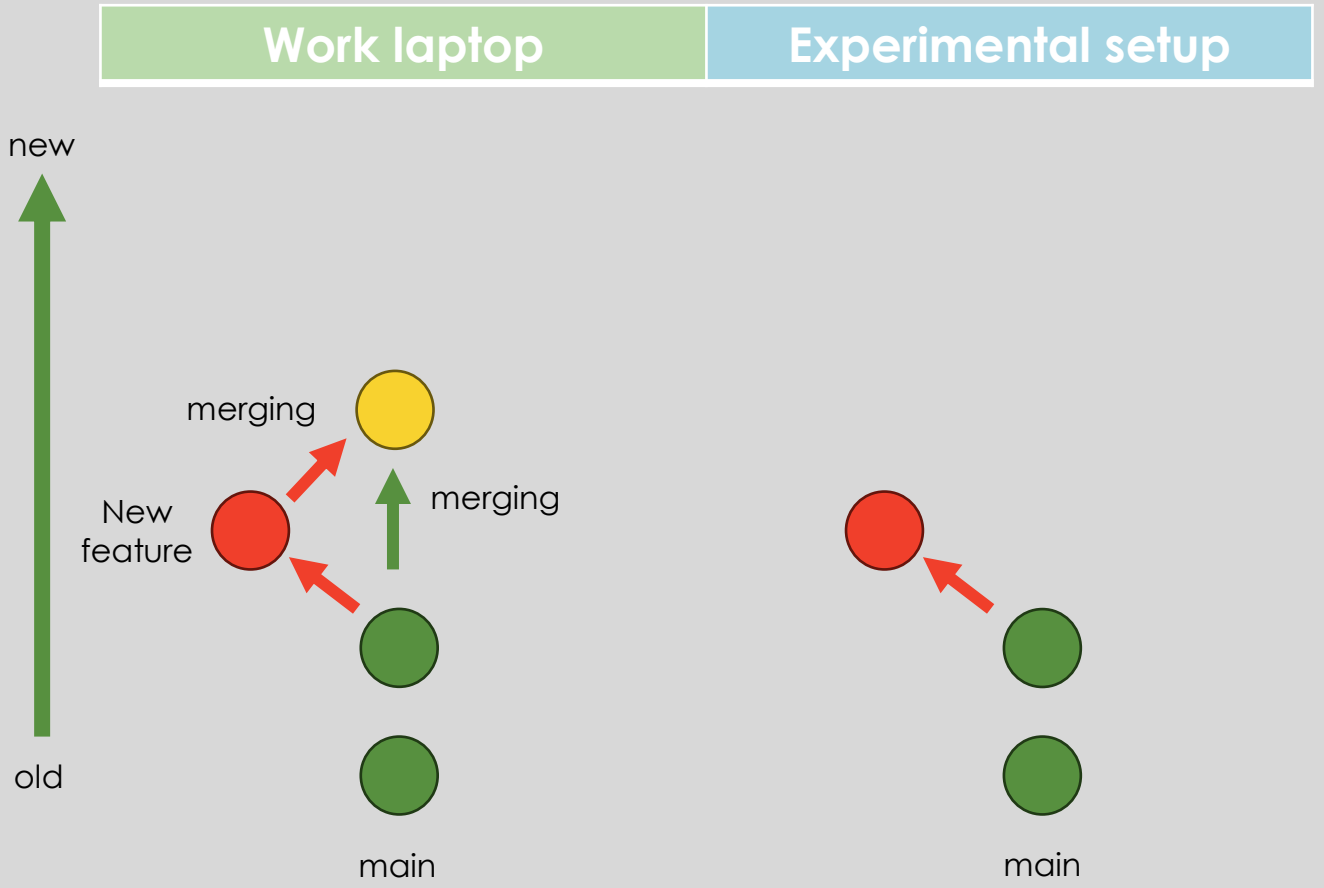
Commands:
$ git checkout main
$ git merge New-feature
$ git branch -d New-feature

- **Merge** and **Delete**
  If you want to implement your changes in "new feature" to the main repository, then you **merge!**
  (Be careful about conflictual changes)

First you switch to you main repository then you use the git merge command

If not needed anymore, you can **Delete** the New-feature branch (git branch -d).

When using Git we talk of a **work/working tree**: a term used to refer to your local repository, including any changes



Commands:
$ touch .gitignore

# Git – Extra stuff

- .gitignore

When sharing your code with others, there are often files or parts of your project, you do not want to share.

For example: the log files, temp files, hidden files, personal files etc…

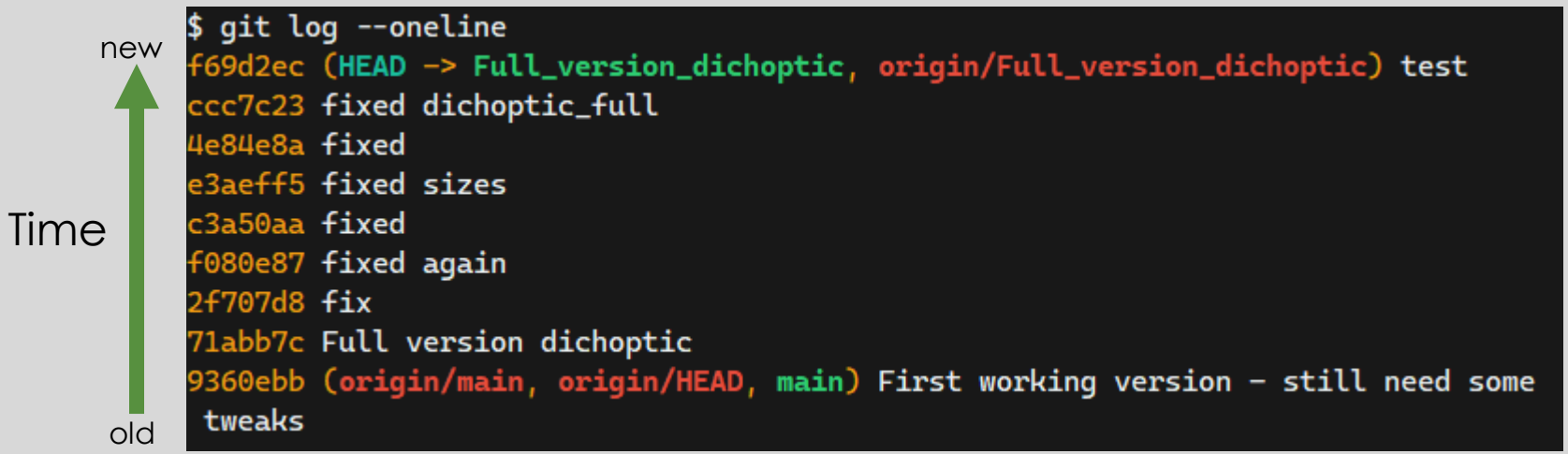
Git can specify which files or parts of your project should be ignored by Git using a .gitignore file.

Git will not track files and folders specified in .gitignore. However, the .gitignore file itself IS tracked by Git.

When the gitignore file is created, you open it using a text editor and specify the files Git should ignore.

# Git – Extra stuff

- Revert, reset
  - First: ALWAYS INDICATE YOUR CHANGES IN THE LOG WHEN YOU COMMIT CHANGES




Do not do like me and be more specific in your commit summaries and descriptions!

# Git – Extra stuff

Commands:
$ git log –oneline
$ git revert *indicate commit*
$ git reset *indicate commit* (e.g. 9a9add8)

- Revert and Reset
  You can see all your commits using the log command.
  Revert is the command we use when we want to take a previous commit and add it as a new commit, keeping the log intact.
  Reset is the command we use when we want to move the repository back to a previous commit, discarding any changes made after that commit.

Example

```
[user@localhost] $ git log --oneline
e56ba1f (HEAD -> master) Revert "Just a regular update, definitely no accidents here..."
52418f7 Just a regular update, definitely no accidents here...
9a9add8 (origin/master) Added .gitignore
81912ba Corrected spelling error
3fdaa5b Merge pull request #1 from w3schools-test/update-readme
836e5bf (origin/update-readme, update-readme) Updated readme for GitHub Branches
daf4f7c (origin/html-skeleton, html-skeleton) Updated index.html with basic meta
facaeae (gh-page/master) Merge branch 'master' of https://github.com/w3schools-test/hello-world
e7de78f Updated index.html. Resized image
5a04b6f Updated README.md with a line about focus
d29d69f Updated README.md with a line about GitHub
e0b6038 merged with hello-world-images after fixing conflicts
1f1584e added new image
dfa79db updated index.html with emergency fix
0312c55 Added image to Hello World
[user@localhost] $ 09f4acd Updated index.html with a new line
[user@localhost] $ 221ec6e First release of Hello World!
```

This is way better!

# Github – Lingua Franca

- Fork and Pull Request

If you want to work on a repository that someone created and release on Github, you can **Fork**.

A copy of the repository will be added to your account.

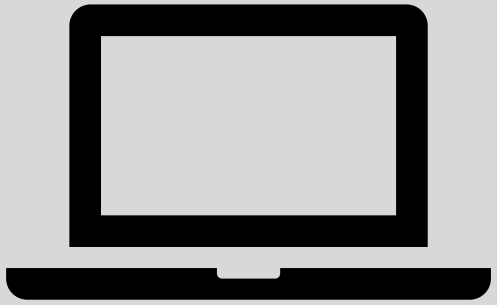
If, after modifying the forked repository (commit changes), you consider your changes are worth to be in the original repository you took it from you can start a "**pull request**".
The owner of the original repository can review your changes and either accept or reject your request.

# Git – Final thoughts

- There is way more to learn about Git! More commands, more uses, how to collaborate etc…
- Now it's your job to try it and make it your own!

If you are a visual person and need a GUI

# A few slides about Github desktop



Work laptop


- Create a Github account

- Install Github Desktop (tool to use Git but with visual interface)
(set up the app, ssh key etc…)

- Create a local repository (Documents/Github/test)

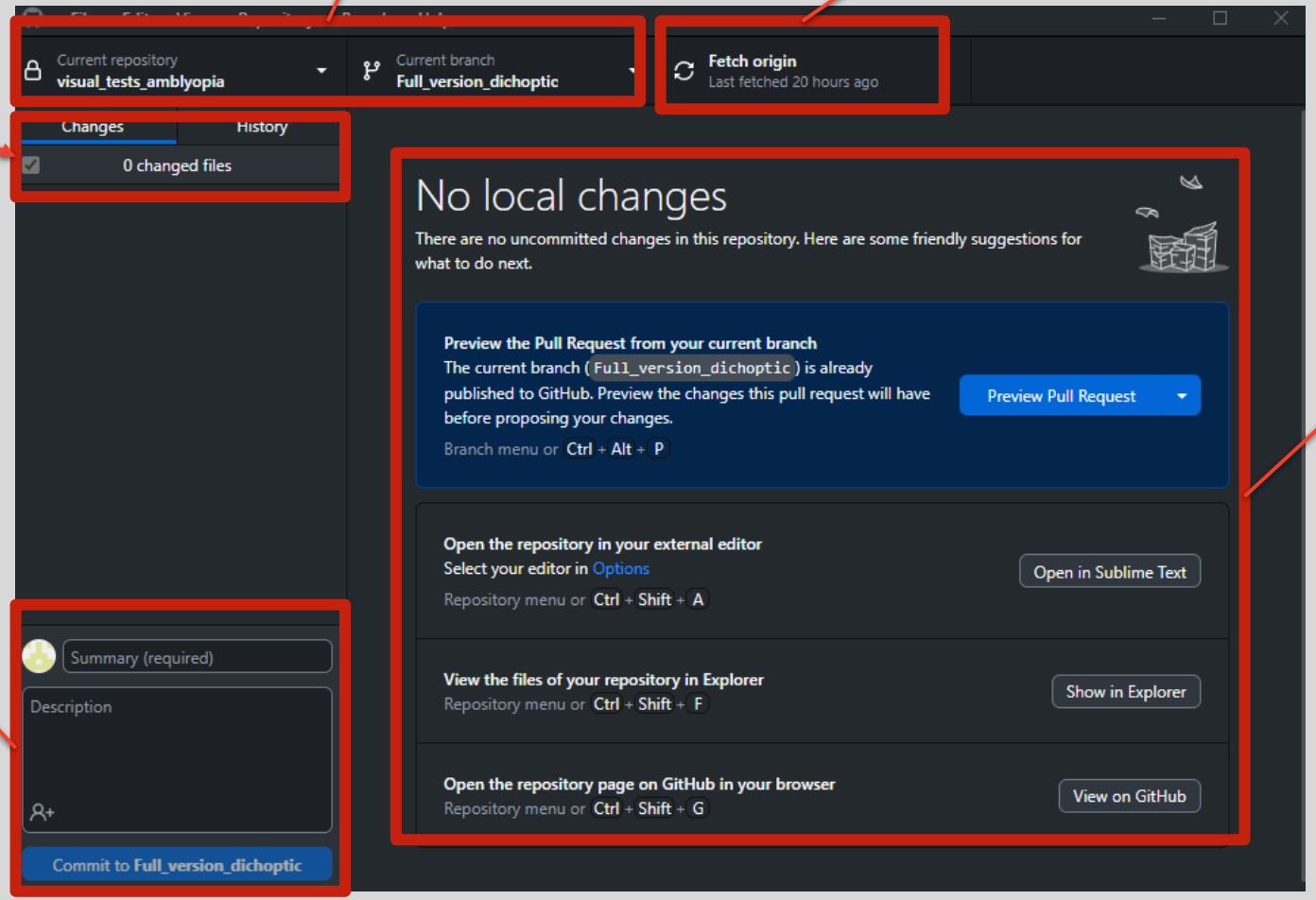
- Add files to this repository (main) and save it

- When the app is paired with your Github account you can
    - push your local repository to an online repository (on Github)
    - pull your online repository to your local repository
    - create and manage branches
    - etc…

Your repository and branch

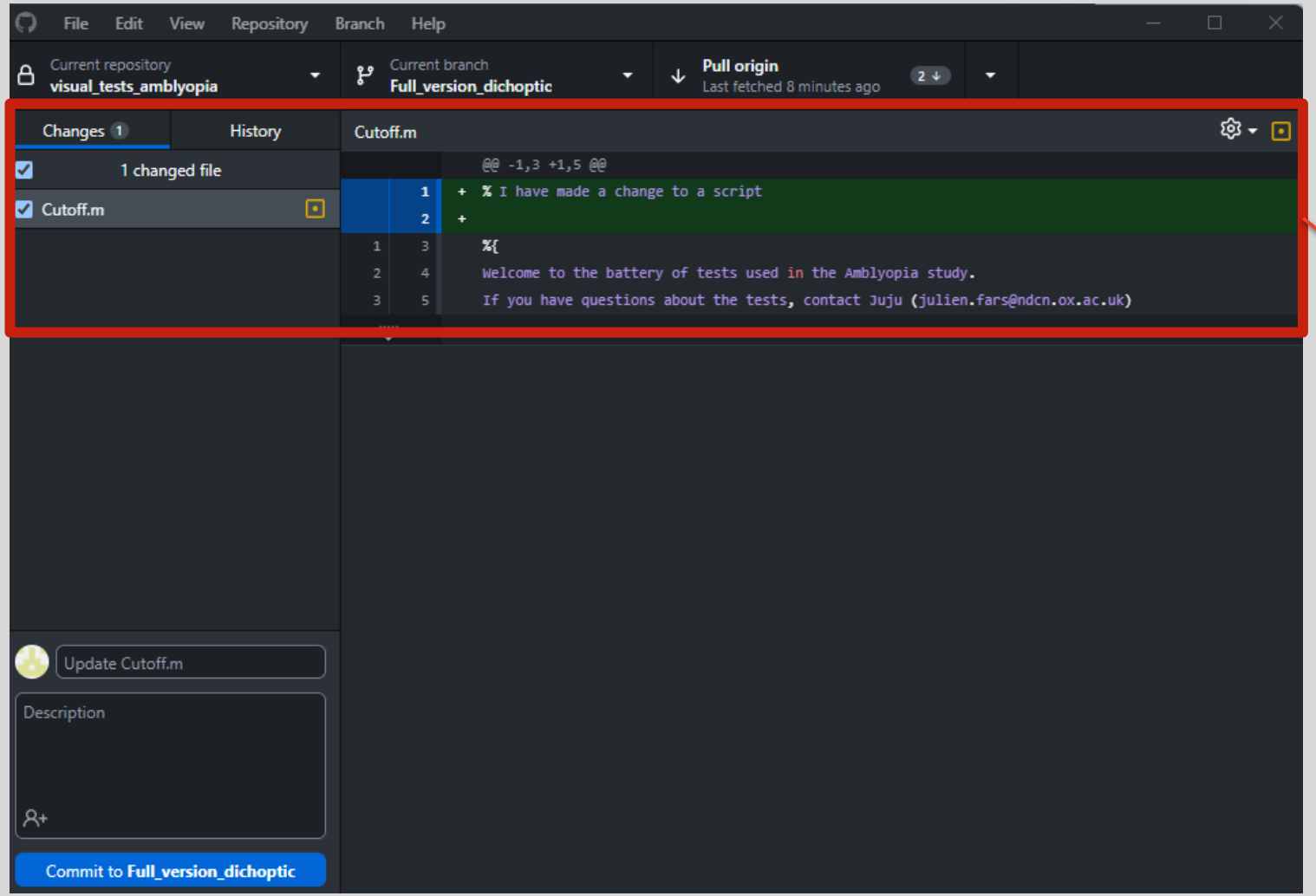
Pull your online repository and 'replace' the local one

See the changes that occurred in each file




- Indicates your local changes
- Integrates your favourite text editor

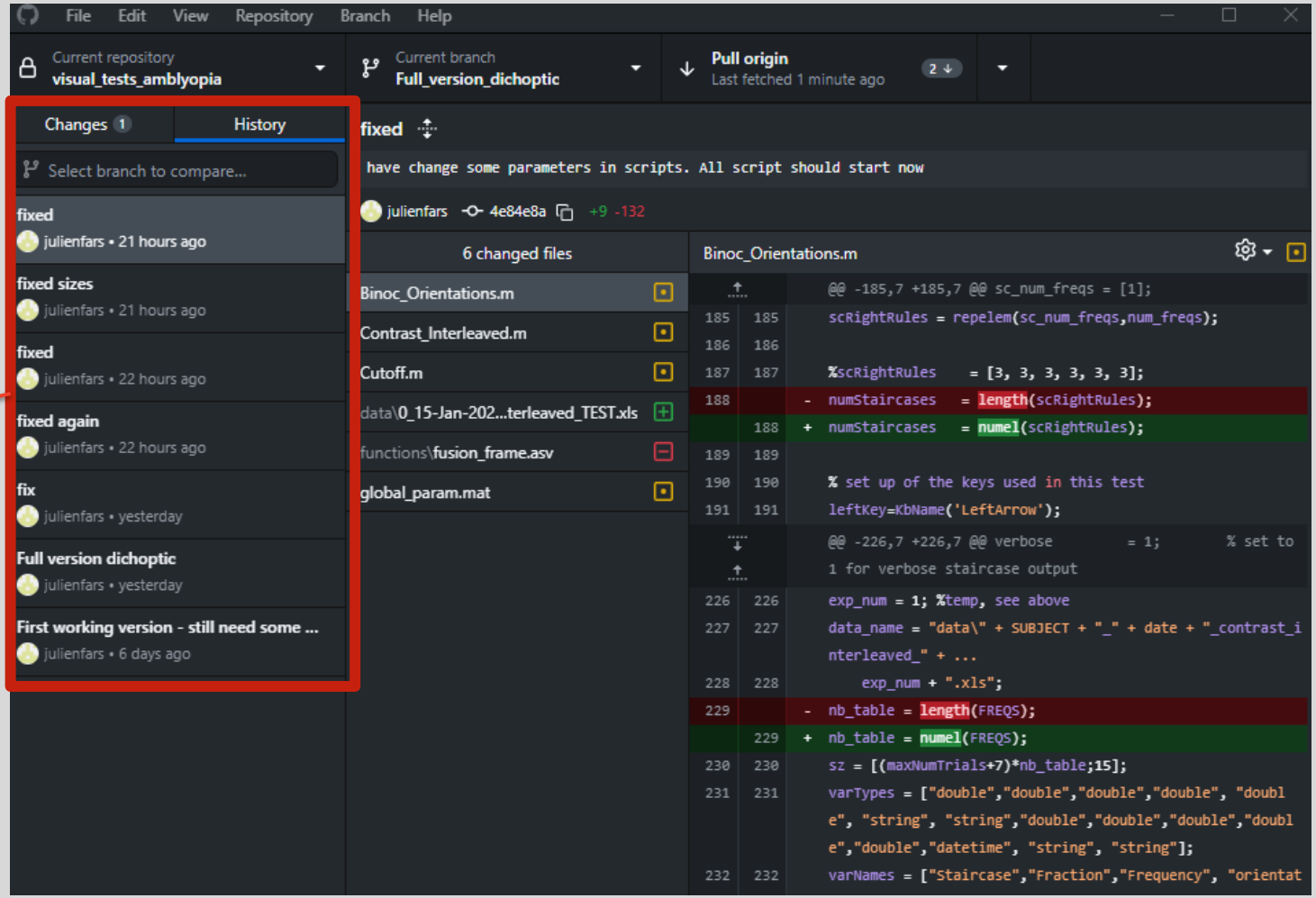
Commit your local change to your online repository




If I make a slight change in a file I immediately see it on Github Desktop

Then I can decide to apply this change online (commit)

The changes history is displayed.
You can select and go back to a previous version



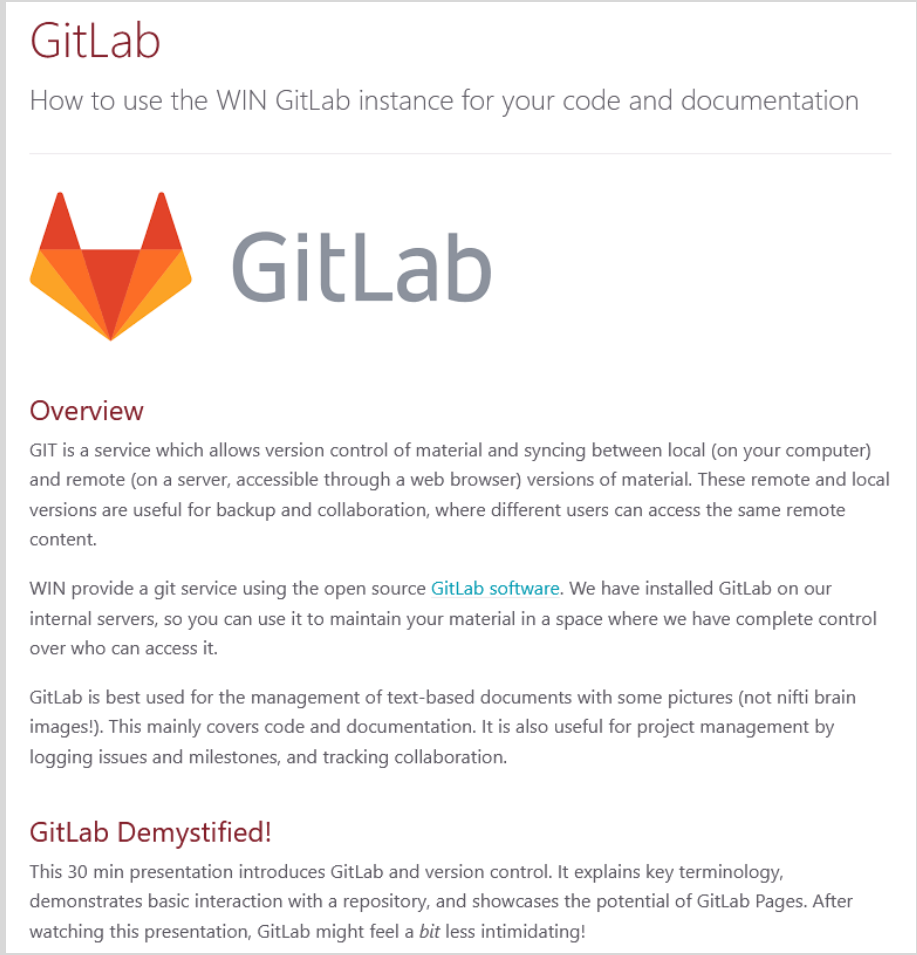
# And for the WIN members

- Can I apply all I saw about Git and Github to Gitlab?

YES

But be careful, Github desktop can only operate 1 account at a time (consider using Git terminal)

- Go to the WIN community pages to start using the WIN Gitlab
- You NEED a WIN computing account (WIN members)
- Follow the tutorial provided by WIN



GitLab

How to use the WIN GitLab instance for your code and documentation

GitLab

## Overview

GIT is a service which allows version control of material and syncing between local (on your computer) and remote (on a server, accessible through a web browser) versions of material. These remote and local versions are useful for backup and collaboration, where different users can access the same remote content.

WIN provide a git service using the open source GitLab software. We have installed GitLab on our internal servers, so you can use it to maintain your material in a space where we have complete control over who can access it.

GitLab is best used for the management of text-based documents with some pictures (not nifti brain images!). This mainly covers code and documentation. It is also useful for project management by logging issues and milestones, and tracking collaboration.

## GitLab Demystified!

This 30 min presentation introduces GitLab and version control. It explains key terminology, demonstrates basic interaction with a repository, and showcases the potential of GitLab Pages. After watching this presentation, GitLab might feel a *bit* less intimidating!

OPEN win COMMUNITY

wellcome centre integrative neuroimaging

# Thank you!

If you have any questions

open@win.ox.ac.uk

## Special thanks to

Cassandra Gould Van Praag

Bernd Taschler

Adina Wagner

Holly Bear

Laurence Hunt

W3schools

Paul McCarthy